BCSE303P - Operating Systems Lab • September 2026

Smart Hospital Resource Management System

An Application-Oriented OS Project Overview Demonstrating CPU Scheduling, Process Control, Mutex Synchronization, Banker's Algorithm & Memory Paging

Havish
Reg. No: 24BYB0057

Shanmukha Simha
Reg. No: 24BDE0123

Guided by: **Dr. PORKODI S** (School of Computer Science Engineering & Information Systems)

Project Overview & Basic Idea

Why We Built This Project

In our OS lab curriculum, concepts like scheduling, mutexes, and deadlocks are usually studied as separate textbook C programs.

We wanted to build a practical system where multiple OS concepts work together to solve a real-world problem.

The Hospital Problem

Hospital emergency wards face strict resource bottlenecks:

- Limited doctors (CPU cores) for consultations
- Limited specialized machines (ICU, MRI, Surgery Rooms)
- Varying patient emergency urgency levels

Real-World Mapping to OS Concepts

Slide 03 / 10

Real-World Hospital Element	Corresponding OS Concept	How It Operates in Code
Patient Admissions	OS Processes & PCB	Tracks PID, priority, burst time, and state (NEW to DONE)
Triage Doctor / Consultation	CPU Scheduling & Preemption	P1 Emergency Trauma preempts lower priority P4 cases on CPU
Shared Medical Machines (ICU, MRI)	Mutex Locks & Semaphores	Locks resource; moves competing patients to WAITING queue
Multi-Machine Surgery Booking	Banker's Algorithm & RAG Graph	Verifies Safe Sequence (`Need = Max - Alloc`) & detects cycle deadlocks
Patient File & Scan Records	Memory Partitioning & Paging	First/Best-Fit RAM allocation + LRU/FIFO virtual page replacement

Concept 1: CPU Priority Scheduling

Slide 04 / 10

4 Priority Triage Levels

- **P1 (Emergency Trauma)**: Highest priority (Heart attack, head injury). Preempts running CPU task immediately.
- **P2 (Urgent Surgery)**: High priority queue placement.
- **P3 (General OPD Ward)**: Standard doctor consultation.
- **P4 (Routine Checkup)**: Lowest priority queue.

Preemption & Aging Mechanism

- **Preemptive Context Switch**: When a P1 patient arrives, the doctor CPU pauses lower priority tasks and handles P1 immediately.
- **Aging Counter**: If a routine P4 patient waits in the queue for >10 ticks, aging logic boosts its priority to prevent starvation.

Concept 2: Process Control Block (PCB)

Slide 05 / 10

PCB Structure

- `PID`: Unique ID (e.g., P101, P102)
- `Priority`: Urgency level (P1 to P4)
- `Burst Time`: Remaining treatment duration
- `Memory Required`: Allocation in KB
- `Resource Claim`: Equipment required (ICU, OR, MRI)
- `Metrics`: Arrival, Wait & Turnaround times

Process 5-State Machine

Each patient process transitions through standard OS lifecycle states:

NEW ➔ READY ➔ RUNNING ➔ WAITING ➔ DONE

If an equipment lock is busy, the process stays in **WAITING** until freed.

Concept 3: Thread Synchronization & Mutexes

Shared Critical Equipment

- **ICU Bed Mutex**: Exclusive single-patient access.
- **Ventilator Mutex**: Critical respiratory support lock.
- **MRI Scanner Mutex**: Diagnostic imaging suite lock.
- **Operating Room Mutex**: Surgical suite critical section.

Preventing Race Conditions

Two patient tasks cannot book the same MRI scanner simultaneously.

Our code uses a Mutex class. When a patient process acquires the lock, status becomes `LOCKED`. Requesting processes move to the **Waiting List** without causing race conditions.

Concept 4: Banker's Algorithm & RAG Graph

Banker's Safety Verification

Before granting multi-resource equipment requests, the algorithm calculates matrices to ensure system safety:

- Maintains `Available`, `Max`, `Allocation`
- Computes `Need = Max - Allocation`
- Verifies that a valid safe sequence exists

Resource Allocation Graph (RAG)

We render an interactive RAG graph on Canvas:

- **Circles** = Patient Processes
- **Squares** = Equipment Resources
- **Solid Arrows** = Held Allocations
- **Dashed Arrows** = Pending Requests
- Auto-detects circular wait deadlocks.

Concept 5: Memory Allocation & Paging

Slide 08 / 10

Main Memory Allocation (RAM)

A 1024 KB memory block is managed to store active patient files:

- **First-Fit**: Allocates in first open space.
- **Best-Fit**: Minimizes internal fragment size.
- **Worst-Fit**: Uses largest available block.
- Adjacent free blocks merge upon discharge.

Virtual Memory Paging

Simulates accessing scan pages into 4 physical frames:

- **LRU (Least Recently Used)**: Evicts oldest unaccessed page.
- **FIFO**: Replaces the oldest loaded page.
- Live counters for Page Hits, Page Faults, and Hit Ratio.

</> System Architecture & Tech Stack

Frontend Dashboard

HTML5 & CSS3 with clean responsive layouts, data tables, Gantt chart, RAG canvas, and console log streamer.

7 JavaScript Modules

Modular ES6 logic (`process_manager.js`, `cpu_scheduler.js`, `banker_rag.js`, `synchronization.js`, `memory_manager.js`, `os_engine.js`, `ui_controller.js`).

Live Simulation Engine

Central clock ticker loop running at 0.8s per tick with step-by-step execution and scenario presets.

Conclusion & Summary

Key Takeaways

- Successfully integrated **5 core Operating System concepts** into a practical, real-world application.
- Demonstrated how scheduling algorithms, mutual exclusion locks, and memory algorithms solve hospital triage bottlenecks.
- Created an interactive dashboard for live testing, Gantt visualization, and deadlock safety verification.